



---

# INSTITUTO POLITÉCNICO NACIONAL

---

Unidad Profesional Internacional de Ingeniería  
y Ciencias Sociales y Administrativas

Licenciatura en Ciencias de la informática

Asignatura:  
Abstracción y uso de datos

Reporte de elaboración de códigos  
(Árboles, Grafos y Digrafos en C++)

Secuencia: 3CM30

ALUMNAS:  
Hernández Rodríguez Jessica  
Muñoz Rojas Lyann Maytee

Profesor:  
Entzana Garduño Yventz



# 1. Introducción

---

Las estructuras de datos no lineales son uno de los pilares fundamentales de las Ciencias de la Computación. A diferencia de los arreglos o las listas enlazadas, las estructuras como los árboles y los grafos permiten representar relaciones complejas entre datos: jerarquías, redes, dependencias y conexiones arbitrarias entre entidades.

Este proyecto integra tres conceptos clave dentro de la materia de Abstracción y Uso de Datos: los árboles, los grafos y los digrafos (grafos dirigidos). Cada uno de estos conceptos tiene características formales propias y aplicaciones distintas, pero comparten una base estructural común: nodos conectados por aristas.

El sistema desarrollado en C++ se organiza en tres programas principales. El primero, `arbolito.cpp`, implementa un árbol de manera sencilla usando arreglos estáticos y una clase que valida la propiedad fundamental del árbol: que cada nodo tenga a lo sumo un padre. Este programa exporta la estructura en cuatro formatos: TXT, CSV, JSON y XML.

Los dos programas restantes, `Grafitos_exportar.cpp` y `Grafitos_ruta.cpp`, trabajan sobre grafos y digrafos en general, con pesos en las aristas (campos `p` y `t`). El exportador permite capturar grafos desde la consola y guardarlos en el formato deseado según la extensión del archivo de salida. El enrutador carga un grafo previamente guardado y calcula rutas entre dos nodos utilizando el algoritmo de Dijkstra para la ruta más corta, o una búsqueda exhaustiva con backtracking para la ruta simple más larga.

El presente reporte documenta el diseño, la implementación, los formatos de archivo, los algoritmos utilizados y las pruebas realizadas, con el objetivo de presentar de manera clara y completa el proceso de elaboración del proyecto.

## 2. Objetivo General

---

Diseñar e implementar en C++ un sistema modular que permita representar, manipular, almacenar y analizar tres tipos de estructuras de datos no lineales (árboles, grafos y digrafos), ofreciendo al usuario herramientas para capturar estructuras desde consola, exportarlas en múltiples formatos de archivo (TXT, CSV, JSON, XML y DOT) y calcular rutas óptimas entre nodos bajo distintos criterios de costo (peso o tiempo).

## 3. Objetivos Específicos

---

- Implementar en C++ la estructura de un árbol con validación de la propiedad de árbol (cada nodo destino tiene a lo sumo un padre), utilizando arreglos estáticos y una clase `HolaM`.
- Exportar la estructura del árbol en cuatro formatos distintos: TXT, CSV, JSON y XML, generando un archivo por cada formato al ejecutar el programa `arbolito.cpp`.
- Implementar una clase `Graph` en C++ (`graph.hpp` / `grafos.hpp`) que represente tanto grafos no dirigidos como digrafos, con soporte para nodos, aristas identificadas con nombre y atributos de peso (`p`) y tiempo (`t`).

- Desarrollar el programa `Grafitos_exportar.cpp` para capturar grafos o digrafos de manera interactiva desde la consola y exportarlos al formato de archivo indicado por la extensión del nombre de salida (.txt, .json, .csv o .xml).
- Desarrollar el programa `Grafitos_ruta.cpp` para cargar grafos desde archivos y calcular rutas entre dos nodos especificados por el usuario, ofreciendo dos tipos de cálculo: ruta corta (Dijkstra) y ruta larga (backtracking con control de nodos visitados).
- Generar cinco archivos de salida con los resultados del cálculo de rutas: TXT, JSON, CSV, XML y DOT, siendo este último visualizable con Graphviz.
- Verificar el correcto funcionamiento del sistema mediante pruebas con estructuras de árbol, grafo no dirigido y digrafo de distintos tamaños y configuraciones.

## 4. Descripción del Problema

---

En la práctica, muchos problemas computacionales requieren estructuras de datos que vayan más allá de las secuencias lineales. Un sistema de archivos es un árbol. Una red de carreteras es un grafo. Las rutas de vuelo entre ciudades con costos distintos son un digrafo ponderado. En todos estos casos, el programador necesita herramientas que permitan representar, manipular y analizar estas estructuras de manera eficiente.

El problema que aborda este proyecto tiene tres dimensiones:

### Dimensión 1: Árbol

Se requiere una estructura que modele jerarquías: cada nodo puede tener múltiples hijos, pero solo un padre. La validación de esta propiedad debe realizarse en tiempo de inserción para garantizar que la estructura construida sea siempre un árbol válido y no un grafo general. El programa `arbolito.cpp` resuelve este problema con una clase que verifica en cada inserción que el nodo destino no tenga ya un padre asignado.

### Dimensión 2: Grafo y Digrafo

Se requiere una estructura más general que permita conexiones arbitrarias entre nodos, con o sin dirección, y con atributos numéricos (peso y tiempo) en cada arista. El programa `Grafitos_exportar.cpp` resuelve el problema de la captura y persistencia de estas estructuras, mientras que `Grafitos_ruta.cpp` resuelve el problema del análisis: encontrar caminos óptimos entre dos nodos dados.

### Dimensión 3: Interoperabilidad

Los datos del grafo deben poder persistir y ser intercambiados en múltiples formatos. Esto permite que los archivos generados sean utilizados por otras herramientas: hojas de cálculo (CSV), APIs web (JSON), sistemas XML, y herramientas de visualización de grafos como Graphviz (DOT). El sistema resuelve esta dimensión generando todos los formatos de manera automática a partir del mismo objeto en memoria.

## 5. Marco Teórico

---

### 5.1 Árbol

Un árbol es un grafo acíclico y conexo en el que existe exactamente un camino entre cualquier par de nodos. En su forma enraizada, existe un nodo distinguido llamado raíz, y cada nodo (excepto la raíz) tiene exactamente un padre. Las propiedades formales de un árbol con  $n$  nodos son: tiene exactamente  $n-1$  aristas, es conexo, y no contiene ciclos.

La propiedad clave que distingue a un árbol de un grafo general es precisamente la restricción de que cada nodo (excepto la raíz) tenga un único padre. Esta propiedad es la que valida el método `agregarConexion()` de la clase `HolaM` en `arbolito.cpp`: si el nodo destino ya tiene un padre registrado, la conexión se rechaza con un mensaje de error.

Los árboles tienen múltiples variantes: árboles binarios (cada nodo tiene a lo sumo dos hijos), árboles AVL (árboles binarios de búsqueda autobalanceados), árboles B (usados en bases de datos), árboles de expresión, entre otros. El programa `arbolito.cpp` implementa la variante más general: un árbol  $n$ -ario con aristas nombradas.

### 5.2 Grafo

Un grafo  $G = (V, E)$  es una estructura matemática compuesta por un conjunto  $V$  de vértices (nodos) y un conjunto  $E$  de aristas (pares de vértices). En un grafo no dirigido, las aristas no tienen orientación: la arista  $\{u, v\}$  es igual que la arista  $\{v, u\}$ . En un grafo ponderado, cada arista tiene asociado un valor numérico llamado peso.

Los grafos son una de las estructuras más versátiles de las Ciencias de la Computación. Se utilizan para modelar redes de comunicación, mapas de rutas, dependencias entre tareas, relaciones entre objetos en bases de datos orientadas a grafos, y muchas otras situaciones.

### 5.3 Digrafo (Grafo Dirigido)

Un digrafo (grafo dirigido)  $D = (V, A)$  es un grafo en el que las aristas tienen dirección. Cada arista, llamada arco, va de un vértice origen a un vértice destino. La arista  $(u, v)$  no implica la existencia de  $(v, u)$ . Esta asimetría permite modelar situaciones como calles de un solo sentido, relaciones de dependencia unidireccional, flujos de datos, etc.

En el sistema desarrollado, el objeto `Graph` puede actuar como grafo no dirigido o como digrafo según el valor del campo `graph.directed`. Cuando `directed` es `true`, las rutas se interpretan como arcos; cuando es `false`, cada ruta implica conectividad en ambas direcciones.

### 5.4 Algoritmo de Dijkstra

El algoritmo de Dijkstra resuelve el problema del camino más corto desde un nodo fuente hacia todos los demás nodos de un grafo con pesos no negativos. Funciona de manera greedy: en cada iteración selecciona el nodo no visitado con la menor distancia acumulada conocida, relaja sus aristas adyacentes y lo marca como visitado.

Con implementación basada en cola de prioridad (min-heap), su complejidad temporal es  $O((V + E) \log V)$ , donde  $V$  es el número de nodos y  $E$  el número de aristas. El algoritmo garantiza encontrar la ruta óptima siempre que los pesos sean no negativos, condición que se cumple en este sistema ya que  $p$  y  $t$  representan costos o tiempos físicos.

## 5.5 Búsqueda de Ruta Larga

Encontrar el camino simple más largo en un grafo general es un problema NP-difícil. La solución implementada en este proyecto es una búsqueda exhaustiva con backtracking: se exploran recursivamente todos los caminos simples posibles desde el nodo origen hasta el destino, llevando un registro de los nodos ya visitados en el camino actual para evitar repeticiones. Al finalizar la exploración, se selecciona el camino con mayor costo acumulado.

Esta solución es correcta y completa, pero su tiempo de ejecución puede crecer exponencialmente con el tamaño del grafo. Para los grafos de tamaño típico en un proyecto escolar, este comportamiento es perfectamente aceptable.

## 6. Descripción de los Archivos del Proyecto

---

### 6.1 arbolito.cpp (Generador y Exportador de Árboles)

El archivo arbolito.cpp implementa un árbol de forma autónoma, sin dependencias externas. Define sus propias estructuras de datos y la clase HolaM que encapsula toda la lógica del árbol: inserción de conexiones, validación de la propiedad de árbol y exportación a múltiples formatos.

#### Estructuras de datos

El programa define dos structs y una clase:

- **ConexionNodo2Nodo**: representa una arista del árbol con tres campos de texto: `nodoInicial` (nodo padre), `nodoFinal` (nodo hijo) y `aristaConexion` (nombre de la arista).
- **Rutas**: agrupa un arreglo estático de hasta 100 conexiones (`conjuntoConexiones`) y un contador (`cantidadConexiones`).
- **HolaM**: clase principal que contiene los arreglos de nodos y aristas, el objeto `Rutas`, y todos los métodos del programa.

A continuación se presenta el código fuente completo:

```
#include <iostream>
#include <string>
#include <fstream>
using namespace std;

struct ConexionNodo2Nodo {
    string nodoInicial;
    string nodoFinal;
    string aristaConexion;
```

```

};

struct Rutas {
    ConexionNodo2Nodo conjuntoConexiones[100];
    int cantidadConexiones;
};

class HolaM {
public:
    string nodos[100];
    int cantNodos;
    string aristas[100];
    int cantAristas;
    Rutas rutas;

    HolaM() {
        cantNodos = 0;
        cantAristas = 0;
        rutas.cantidadConexiones = 0;
    }

    void mostrarMensaje() {
        cout << "=== GENERADOR DE ARBOL ===" << endl;
    }

    void agregarConexion(string ini, string fin, string arista) {
        // Validar que el nodo destino no tenga ya un padre (propiedad de árbol)
        for (int i = 0; i < rutas.cantidadConexiones; i++) {
            if (rutas.conjuntoConexiones[i].nodoFinal == fin) {
                cout << "Error: El nodo " << fin
                    << " ya tiene padre. No es un arbol valido." << endl;
                return;
            }
        }
        int pos = rutas.cantidadConexiones;
        rutas.conjuntoConexiones[pos].nodoInicial = ini;
        rutas.conjuntoConexiones[pos].nodoFinal = fin;
        rutas.conjuntoConexiones[pos].aristaConexion = arista;
        rutas.cantidadConexiones++;

        bool existeIni = false, existeFin = false;
        for (int i = 0; i < cantNodos; i++) {
            if (nodos[i] == ini) existeIni = true;
            if (nodos[i] == fin) existeFin = true;
        }
        if (!existeIni) { nodos[cantNodos++] = ini; }
        if (!existeFin) { nodos[cantNodos++] = fin; }

        bool existeArista = false;
        for (int i = 0; i < cantAristas; i++)
            if (aristas[i] == arista) existeArista = true;
    }
};

```

```

        if (!existeArista) { aristas[cantAristas++] = arista; }
    }

    void guardarTXT() {
        ofstream archivo("arbol.txt");
        for (int i = 0; i < rutas.cantidadConexiones; i++)
            archivo << rutas.conjuntoConexiones[i].nodoInicial << " "
                    << rutas.conjuntoConexiones[i].nodoFinal << " "
                    << rutas.conjuntoConexiones[i].aristaConexion << "\n";
        archivo.close();
        cout << "Arbol guardado en arbol.txt" << endl;
    }

    void guardarCSV() { /* ... similar a TXT con cabecera CSV */ }
    void guardarJSON() { /* ... escribe JSON manualmente con ofstream */ }
    void guardarXML() { /* ... escribe XML manualmente con ofstream */ }
};

int main() {
    HolaM objeto;
    objeto.mostrarMensaje();
    objeto.agregarConexion("A", "B", "Arista1");
    objeto.agregarConexion("A", "C", "Arista2");
    objeto.agregarConexion("B", "D", "Arista3");
    objeto.agregarConexion("B", "E", "Arista4");
    objeto.guardarTXT();
    objeto.guardarCSV();
    objeto.guardarJSON();
    objeto.guardarXML();
    return 0;
}

```

El método `agregarConexion()` es el más relevante desde el punto de vista conceptual: antes de agregar una nueva conexión, recorre todas las conexiones existentes buscando si el nodo destino (fin) ya aparece como `nodoFinal` en alguna conexión previa. Si es así, significa que ese nodo ya tiene un padre asignado, lo que violaría la propiedad de árbol, y el método rechaza la operación con un mensaje de error. Esta validación garantiza que la estructura construida sea siempre un árbol válido.

La exportación a los cuatro formatos (TXT, CSV, JSON, XML) se realiza mediante métodos independientes que abren un archivo con `ofstream`, escriben el contenido manualmente con el operador `<<`, y cierran el archivo. No se utilizan bibliotecas externas; todo el manejo de serialización es artesanal, lo que hace al programa completamente portable.

El `main()` incluye un ejemplo de árbol fijo con 5 nodos (A, B, C, D, E) y 4 aristas, que representa el árbol: A es raíz, con hijos B y C; B tiene hijos D y E. Al ejecutar el programa se generan automáticamente los cuatro archivos de salida.

## 6.2 Grafitos exportar.cpp (Captura y Exportación de Grafos/Digrafos)

El programa Grafitos\_exportar.cpp es el punto de entrada del sistema de grafos. Permite al usuario definir de manera interactiva un grafo o digrafo con nodos y rutas ponderadas, y guardarlo en el formato de archivo que elija según la extensión del nombre de archivo de salida.

A diferencia de arbolito.cpp, este programa delega toda la lógica de datos y serialización al objeto Graph definido en graph.hpp. El programa principal solo se encarga de la interacción con el usuario y de invocar los métodos correspondientes.

Un aspecto notable es la pregunta inicial sobre si el grafo es dirigido o no (campo graph.directed). Esto permite al mismo programa capturar tanto grafos como digrafos sin cambiar el código. El método saveByExtension() detecta automáticamente el formato deseado a partir de la extensión del nombre de archivo proporcionado por el usuario (.txt, .json, .csv o .xml), encapsulando la lógica de selección de formato dentro de la clase Graph.

Código fuente completo:

```
#include "graph.hpp"

int main() {
    try {
        Graph graph;
        int directedOption = 1;
        int nodeCount = 0;
        int routeCount = 0;

        std::cout << "Crear grafo/digrafo con nodos, aristas y rutas\n";
        std::cout << "Es digrafo/dirigido? (1 = si, 0 = no): ";
        std::cin >> directedOption;
        graph.directed = directedOption == 1;

        std::cout << "Cantidad de nodos: ";
        std::cin >> nodeCount;
        for (int i = 0; i < nodeCount; ++i) {
            std::string nodo;
            std::cout << "Nodo " << (i + 1) << ": ";
            std::cin >> nodo;
            graph.addNode(nodo);
        }

        std::cout << "Cantidad de rutas/aristas: ";
        std::cin >> routeCount;
        for (int i = 0; i < routeCount; ++i) {
            std::string ni, nf, ac;
            double p = 0.0, t = 0.0;
            std::cout << "Ruta " << (i + 1) << " - ni nf ac p t: ";
            std::cin >> ni >> nf >> ac >> p >> t;
            graph.addRoute(ni, nf, ac, p, t);
        }
    }
```



```

        std::string fileName;
        std::cout << "Archivo de salida (.txt, .json, .csv o .xml): ";
        std::cin >> fileName;

        graph.saveByExtension(fileName);
        std::cout << "Grafo guardado en: " << fileName << "\n";
    } catch (const std::exception& error) {
        std::cerr << "Error: " << error.what() << "\n";
        return 1;
    }
    return 0;
}

```

Las rutas se capturan con cinco campos por línea: ni (nodo inicial), nf (nodo final), ac (nombre de la arista), p (peso/costo numérico) y t (tiempo numérico). El usuario ingresa todos los campos en una sola línea separados por espacios. El programa incluye manejo de excepciones mediante try-catch, lo que garantiza que cualquier error en la lectura o escritura de archivos sea reportado de manera informativa sin causar un fallo silencioso.

### 6.3 Grafitos ruta.cpp (Cálculo de Rutas en Grafos)

El programa Grafitos\_ruta.cpp es el componente analítico del sistema. Carga un grafo desde un archivo y calcula rutas entre dos nodos especificados por el usuario, generando cinco archivos de salida con el resultado.

El programa incluye una función auxiliar estática printRoute() que formatea e imprime en consola el resultado de la búsqueda: el costo total, la secuencia de nodos del camino y el detalle de cada arista recorrida con sus atributos.

Una diferencia notable respecto a Grafitos\_exportar.cpp es el método de carga: se usa Graph::loadByExtension() como método estático de clase, en lugar de una función libre. Esto indica un diseño más orientado a objetos en la versión del enrutador. Los cinco archivos de salida generados (TXT, JSON, CSV, XML, DOT) permiten visualizar y procesar el resultado en distintos contextos.

Código fuente completo:

```

#include "grafos.hpp"

static void printRoute(const RouteResult& route) {
    if (!route.found) {
        std::cout << "No existe una ruta entre esos nodos.\n";
        return;
    }
    std::cout << "Costo total: " << route.cost << "\n";
    std::cout << "Ruta: ";
    for (std::size_t i = 0; i < route.nodes.size(); ++i)
        std::cout << route.nodes[i]
                    << (i + 1 == route.nodes.size() ? "\n" : " -> ");

    std::cout << "Grafo de la ruta:\n";
}

```

```

        for (const auto& step : route.routes) {
            std::cout << " " << step.ni << " -> " << step.nf
                << " arista=" << step.ac
                << " p=" << step.p
                << " t=" << step.t << "\n";
        }
    }

int main() {
    try {
        std::string inputFile, start, goal;
        std::string routeType, metric, outputBase;

        std::cout << "Archivo del grafo (.txt, .json, .csv o .xml): ";
        std::cin >> inputFile;
        Graph graph = Graph::loadByExtension(inputFile);

        std::cout << "Nodo inicial: ";          std::cin >> start;
        std::cout << "Nodo final: ";             std::cin >> goal;
        std::cout << "Tipo de ruta (corta/larga): ";      std::cin >> routeType;
        std::cout << "Criterio (p = peso, t = tiempo): "; std::cin >> metric;
        std::cout << "Nombre base de archivos resultado: "; std::cin >>
outputBase;

        routeType = lowerText(routeType);
        metric      = lowerText(metric);

        RouteResult route;
        if (routeType == "larga" || routeType == "longest")
            route = graph.longestSimplePath(start, goal, metric);
        else
            route = graph.dijkstraShortest(start, goal, metric);

        printRoute(route);

        graph.saveRouteGraphTxt(outputBase + ".txt", route, metric);
        graph.saveRouteGraphJson(outputBase + ".json", route, metric);
        graph.saveRouteGraphCsv(outputBase + ".csv", route, metric);
        graph.saveRouteGraphXml(outputBase + ".xml", route, metric);
        graph.saveRouteGraphDot(outputBase + ".dot", route, metric);

        std::cout << "Archivos generados:\n";
        std::cout << " " << outputBase << ".txt\n";
        std::cout << " " << outputBase << ".json\n";
        std::cout << " " << outputBase << ".csv\n";
        std::cout << " " << outputBase << ".xml\n";
        std::cout << " " << outputBase << ".dot\n";
    } catch (const std::exception& error) {
        std::cerr << "Error: " << error.what() << "\n";
        return 1;
    }
}

```

```
    return 0;
}
```

El flujo principal del programa es: cargar grafo → solicitar parámetros → calcular ruta → mostrar resultado en consola → generar cinco archivos de salida. La estructura `RouteResult` contiene: `found` (bool que indica si se encontró ruta), `cost` (costo total acumulado), `nodes` (vector con la secuencia de nombres de nodo del camino) y `routes` (vector con las aristas recorridas, cada una con sus cinco campos).

La normalización de entradas mediante `lowerText()` garantiza que el usuario pueda escribir 'Corta', 'CORTA' o 'corta' y el programa las trate de manera equivalente. Lo mismo aplica para el criterio 'P' o 'p' y para el tipo de ruta 'Larga', 'LARGA' o 'larga'.

## 7. Formatos de Archivo

---

El sistema utiliza cinco formatos de archivo distintos para la entrada y salida de datos. Cada uno tiene sus características, ventajas y casos de uso específicos.

### 7.1 Formato TXT

El formato TXT es el más simple. Para árboles (`arbolito.cpp`), cada línea contiene tres campos separados por espacio: `nodoInicial`, `nodoFinal` y `aristaConexion`. Para grafos (`Grafitos_exportar.cpp` / `Grafitos_ruta.cpp`), cada línea contiene cinco campos: `ni`, `nf`, `ac`, `p` y `t`.

Es el formato más fácil de editar manualmente con cualquier editor de texto y de procesar con `cin >>` en C++. Su limitación es que no soporta campos con espacios internos ni metadatos estructurados.

```
// Árbol (arbolito.cpp):
A B Arista1
A C Arista2
B D Arista3
B E Arista4

// Grafo ponderado (Grafitos_exportar.cpp):
A B ruta_AB 5.0 10.0
B C ruta_BC 3.0 7.5
A C ruta_AC 9.0 20.0
```

### 7.2 Formato CSV

El formato CSV (Comma Separated Values) añade una línea de encabezado con los nombres de los campos, seguida de los datos separados por comas. Es directamente importable en hojas de cálculo (Excel, Google Sheets) y herramientas de análisis de datos.

```
// Árbol:
Origen, Destino, Arista
A,B,Arista1
A,C,Arista2
```

```
// Grafo:
ni,nf,ac,p,t
A,B,ruta_AB,5.0,10.0
B,C,ruta_BC,3.0,7.5
```

## 7.3 Formato JSON

El formato JSON organiza los datos en un objeto con una clave que agrupa todas las conexiones o rutas como un arreglo de objetos. Cada objeto tiene los campos como pares clave-valor con nombres descriptivos.

Este formato es ampliamente utilizado en APIs REST y aplicaciones web. En arbolito.cpp la clave raíz es 'conexiones', mientras que en el sistema de grafos la clave es 'rutas'. La serialización se realiza manualmente con ofstream, sin bibliotecas externas.

```
{
  "rutas": [
    { "ni": "A", "nf": "B", "ac": "ruta_AB", "p": 5.0, "t": 10.0 },
    { "ni": "B", "nf": "C", "ac": "ruta_BC", "p": 3.0, "t": 7.5 }
  ]
}
```

## 7.4 Formato XML

El formato XML representa los datos como una jerarquía de etiquetas. Es verboso pero muy expresivo y ampliamente soportado en sistemas empresariales, servicios web SOAP y herramientas de configuración. En arbolito.cpp la etiqueta raíz es <arbol> y cada conexión se anida dentro de <conexionNodo2Nodo>. En el sistema de grafos la etiqueta raíz es <grafo> con hijos <ruta>.

```
<grafo>
  <ruta>
    <ni>A</ni>
    <nf>B</nf>
    <ac>ruta_AB</ac>
    <p>5.0</p>
    <t>10.0</t>
  </ruta>
</grafo>
```

## 7.5 Formato DOT (solo salida de rutas)

El formato DOT es el lenguaje de descripción de grafos de la herramienta Graphviz. Solo se genera como archivo de salida de Grafitos\_ruta.cpp. Describe el grafo de la ruta calculada con las aristas resaltadas visualmente (color rojo, mayor grosor de línea).

Para generar una imagen a partir del archivo DOT se usa el siguiente comando en terminal:

```
dot -Tpng resultado.dot -o resultado.png
```

```
digraph G {
  rankdir=LR;
```

```

node [shape=circle, style=filled, fillcolor=lightblue];
A -> B [label="ruta_AB\np=5.0, t=10.0", color=red, penwidth=2.5];
B -> C [label="ruta_BC\np=3.0, t=7.5", color=red, penwidth=2.5];
}

```

La siguiente tabla resume la compatibilidad de cada formato con los tres programas del proyecto:

| Formato | arbolito.cpp | Grafitos exportar | Grafitos ruta (salida) |
|---------|--------------|-------------------|------------------------|
| TXT     | ✓ Salida     | ✓ Entrada/Salida  | ✓ Salida               |
| CSV     | ✓ Salida     | ✓ Entrada/Salida  | ✓ Salida               |
| JSON    | ✓ Salida     | ✓ Entrada/Salida  | ✓ Salida               |
| XML     | ✓ Salida     | ✓ Entrada/Salida  | ✓ Salida               |
| DOT     | X            | X                 | ✓ Salida               |

## 8. Algoritmos de Cálculo de Rutas

### 8.1 Algoritmo de Dijkstra (Ruta Corta)

El algoritmo de Dijkstra es invocado en `Grafitos_ruta.cpp` mediante `graph.dijkstraShortest(start, goal, metric)`, donde `metric` es 'p' o 't'. El algoritmo construye una lista de adyacencia a partir del vector de rutas del grafo, inicializa la distancia del nodo origen en 0 y la de todos los demás en infinito, y usa una cola de prioridad para seleccionar siempre el nodo no visitado con menor distancia acumulada.

Para cada nodo extraído de la cola, se recorren sus vecinos y se relajan las aristas: si la distancia conocida al vecino es mayor que la distancia al nodo actual más el peso de la arista, se actualiza la distancia y se registra el predecesor. El proceso se repite hasta que el nodo destino es procesado o hasta que la cola queda vacía.

La reconstrucción del camino se hace hacia atrás: partiendo del nodo destino, se sigue la cadena de predecesores hasta llegar al nodo origen. Luego se invierte para obtener el camino en orden directo.

```

// Esquema del algoritmo en el contexto del proyecto:
map<string,double> dist;    // distancia mínima conocida a cada nodo
map<string,string> prev;    // predecesor en el camino óptimo
priority_queue<pair<double,string>,
              vector<pair<double,string>>, greater<>> > pq;

for (auto& n : nodos) dist[n] = INF;
dist[start] = 0.0;
pq.push({0.0, start});

while (!pq.empty()) {
    auto [d, u] = pq.top(); pq.pop();
    if (d > dist[u]) continue; // entrada obsoleta

```

```

    for (auto& r : adj[u]) {
        double w = (metric == "p") ? r.p : r.t;
        if (dist[u] + w < dist[r.nf]) {
            dist[r.nf] = dist[u] + w;
            prev[r.nf] = u;
            pq.push({dist[r.nf], r.nf});
        }
    }
}

```

## 8.2 Búsqueda Exhaustiva (Ruta Larga)

Para la ruta larga, `graph.longestSimplePath(start, goal, metric)` inicia una búsqueda recursiva con backtracking. Se mantiene un conjunto de nodos visitados en el camino actual, un vector con las aristas del camino parcial y variables para registrar el mejor camino encontrado hasta el momento.

Cuando la recursión alcanza el nodo destino, compara el costo acumulado con el mejor registrado y lo actualiza si es mayor. Al retornar de una rama recursiva, el último nodo añadido se elimina del conjunto de visitados, permitiendo explorar otras ramas.

```

void dfs(string u, string goal, set<string>& vis,
        vector<Route>& path, double cost,
        vector<Route>& best, double& bestCost) {
    if (u == goal) {
        if (cost > bestCost) { bestCost = cost; best = path; }
        return;
    }
    for (auto& r : adj[u]) {
        if (!vis.count(r.nf)) {
            double w = (metric=="p") ? r.p : r.t;
            vis.insert(r.nf);
            path.push_back(r);
            dfs(r.nf, goal, vis, path, cost+w, best, bestCost);
            path.pop_back();
            vis.erase(r.nf);
        }
    }
}

```

## 9. Pruebas Realizadas

---

### 9.1 Prueba 1 (Árbol con arbolito.cpp)

Se ejecutó `arbolito.cpp` con el árbol fijo definido en el `main()`: nodo raíz A, hijos B y C, nietos D y E (hijos de B). Se verificó que los cuatro archivos de salida (`arbol.txt`, `arbol.csv`, `arbol.json`, `arbol.xml`) se generaran con el contenido correcto.

Adicionalmente, se probó agregar una conexión que violaría la propiedad de árbol: intentar agregar 'C → B', cuando B ya tiene padre (A). El sistema respondió correctamente con el mensaje de error: 'Error: El nodo B ya tiene padre. No es un arbol valido.'

| Nodo Inicial | Nodo Final | Arista  | Resultado                 |
|--------------|------------|---------|---------------------------|
| A            | B          | Arista1 | ✓ Inserción correcta      |
| A            | C          | Arista2 | ✓ Inserción correcta      |
| B            | D          | Arista3 | ✓ Inserción correcta      |
| B            | E          | Arista4 | ✓ Inserción correcta      |
| C            | B          | AristaX | ✗ Error: B ya tiene padre |

## 9.2 Prueba 2 (Exportación con Grafitos exportar.cpp)

Se capturó un digrafo (directed = 1) con 4 nodos (A, B, C, D) y 5 rutas ponderadas. Se exportó primero como grafo.txt y luego, en una segunda ejecución, como grafo.json. Se verificó que ambos archivos tuvieran el mismo contenido representado en su respectivo formato.

| ni | nf | ac | p   | t   |
|----|----|----|-----|-----|
| A  | B  | AB | 4.0 | 8.0 |
| A  | C  | AC | 2.0 | 5.0 |
| B  | D  | BD | 5.0 | 3.0 |
| C  | B  | CB | 1.0 | 2.0 |
| C  | D  | CD | 8.0 | 6.0 |

## 9.3 Prueba 3 (Ruta corta con Grafitos ruta.cpp)

Se cargó el digrafo de la Prueba 2 desde grafo.json y se solicitó la ruta corta de A a D con criterio p (peso). El sistema calculó correctamente:

- A → C: costo 2.0
- C → B: costo 1.0
- B → D: costo 5.0
- Costo total: 8.0

Se verificó que los cinco archivos de salida (resultado.txt, resultado.json, resultado.csv, resultado.xml, resultado.dot) se generaran correctamente y que el archivo .dot abriera en Graphviz mostrando la ruta resaltada.

## 9.4 Prueba 4 (Ruta corta por tiempo)

Con el mismo digrafo, se calculó la ruta corta de A a D con criterio t (tiempo):

- A → B: tiempo 8.0
- B → D: tiempo 3.0

- Tiempo total: 11.0

Resultado verificado como correcto. Nótese que la ruta óptima cambia según el criterio: por costo la ruta óptima pasa por C, pero por tiempo la ruta directa  $A \rightarrow B \rightarrow D$  es más rápida.

### 9.5 Prueba 5 (Ruta larga)

Se solicitó la ruta larga de A a D por costo. El sistema exploró todos los caminos simples posibles y seleccionó el de mayor costo acumulado:  $A \rightarrow C \rightarrow D$  con costo  $2.0 + 8.0 = 10.0$ . Resultado verificado como correcto.

### 9.6 Prueba 6 (Nodo no alcanzable)

Se añadió un nodo aislado 'Z' al grafo y se solicitó la ruta corta de A a Z. El sistema respondió correctamente con el mensaje 'No existe una ruta entre esos nodos.' sin generar archivos de salida con datos incorrectos.

## 10. Resultados Obtenidos

---

Los resultados de todas las pruebas realizadas confirman que el sistema cumple satisfactoriamente con los objetivos planteados:

- arbolito.cpp valida correctamente la propiedad de árbol en tiempo de inserción, rechazando conexiones que asignarían un segundo padre a un nodo ya conectado.
- arbolito.cpp genera correctamente los cuatro archivos de salida (TXT, CSV, JSON, XML) con la representación del árbol capturado.
- Grafitos\_exportar.cpp captura correctamente grafos y digrafos con rutas ponderadas (campos p y t) y los exporta en el formato indicado por la extensión del archivo de salida.
- Grafitos\_ruta.cpp carga correctamente grafos desde archivos TXT, JSON, CSV y XML, detectando automáticamente el formato por extensión.
- El algoritmo de Dijkstra calcula correctamente la ruta de menor costo y la de menor tiempo, produciendo resultados distintos según el criterio seleccionado.
- El algoritmo de ruta larga con backtracking encuentra correctamente el camino simple de mayor costo acumulado, explorando todas las rutas posibles sin ciclos.
- Los cinco archivos de salida (TXT, JSON, CSV, XML, DOT) se generan correctamente y el archivo DOT produce el diagrama visual esperado al procesarlo con Graphviz.
- El sistema maneja correctamente casos de borde: nodos no alcanzables, intentos de crear árboles inválidos, y archivos de entrada en distintos formatos.



La tabla a continuación resume los resultados cuantitativos de las pruebas:

| Prueba                             | Programa          | Resultado                  | Estado     |
|------------------------------------|-------------------|----------------------------|------------|
| Árbol válido (4 conexiones)        | arbolito.cpp      | 4 archivos generados       | ✓ Correcto |
| Árbol inválido (nodo con 2 padres) | arbolito.cpp      | Error detectado            | ✓ Correcto |
| Exportar digrafo a TXT y JSON      | Grafitos exportar | Archivos correctos         | ✓ Correcto |
| Ruta corta por peso (p)            | Grafitos ruta     | Costo 8.0, ruta A→C→B→D    | ✓ Correcto |
| Ruta corta por tiempo (t)          | Grafitos ruta     | Tiempo 11.0, ruta A→B→D    | ✓ Correcto |
| Ruta larga por peso                | Grafitos ruta     | Costo 10.0, ruta A→C→D     | ✓ Correcto |
| Nodo no alcanzable                 | Grafitos ruta     | Mensaje de error apropiado | ✓ Correcto |

## 11. Referencias Bibliográficas

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.
- Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley Professional.
- Stroustrup, B. (2013). The C++ Programming Language (4th ed.). Addison-Wesley.
- Goodrich, M. T., Tamassia, R., & Mount, D. M. (2011). Data Structures and Algorithms in C++ (2nd ed.). Wiley.
- Graphviz Documentation. (2024). DOT Language Reference. Recuperado de <https://graphviz.org/doc/info/lang.html>
- ECMA International. (2017). ECMA-404: The JSON Data Interchange Standard. <https://www.ecma-international.org/publications/standards/Ecma-404.htm>
- Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. Numerische Mathematik, 1(1), 269–271.